MINI PROJECT REPORT

ON

“ PROJECT TITLE ”

SUBMITTED BY

PRN	NAME OF STUDENTS
1012411149	Aditya Deo
1012411150	Rounak Singh
1012411154	Viraj Shah
1012411155	Nikhil Karad

2025-26

Second Year Computer Science & Engineering (SY CSE)

Division: B, Batch: B2, Semester - II

Department of Computer Engineering and Technology
School of Engineering and Technology
DES Pune University, Pune.
2025-2026
Program: B. Tech Computer Science & Engineering
School of Engineering and Technology
DES Pune University, Pune

CERTIFICATE

This is to certify that student _____, PRN._____ is studying in **SY CSE (Division: B Batch: B1/B2)** course in SEM II and he/she has successfully completed and submitted the **Mini Project Report under the subject Data Structures - II** , entitled "_____". This study is a partial fulfillment of the degree of Bachelor of Technology in Computer Science and Engineering of **DES Pune University, Pune** during the academic year **2025-2026.**

Dr. SHRIDA KALAMKAR — Subject Teacher

Dr. PRACHI JOSHI — HoS

ABSTRACT

The exponential growth of the internet has led to increasingly complex and lengthy URLs that pose challenges in sharing, readability, and tracking. This project presents **Mini Bit.ly**, a simplified URL shortening system inspired by production-grade services such as Bit.ly and TinyURL, developed as a practical application of the Data Structures concepts that we have learned throughout the semester.

Our system accepts a long URL as input and returns a compact, unique short code using **Base62 encoding**, which is applied over a globally auto-incrementing integer counter. This counter-based approach guarantees zero collisions, with a 6-character key space yielding over **56.8 billion unique combinations** - sufficient for over 155 years at one million new URLs per day.

The project implements several core data structures: a **dual HashMap** for O(1) URL storage and deduplication; a **Max-Heap (Priority Queue)** for tracking the most frequently accessed URLs; an **AVL Tree** for alphabetically organizing URLs by category; a **Doubly Linked List** combined with a HashMap to implement an **LRU Cache** for fast redirect resolution; and the **Base62 encoding algorithm** for collision-free short code generation.

The system is built in Java, following **Object-Oriented Principles** including encapsulation, inheritance, and polymorphism, and incorporates exception handling for robustness. The architecture mirrors industry-standard designs; which we have studied via various **research papers / trusted breakdowns**. With the in-memory HashMap serving as the equivalent of the Redis + MySQL stack used by real-world services. Future enhancements include migration to persistent MySQL storage, Redis-based distributed caching, and a Key Generation Service for horizontal scaling.

We have included extensive breakdowns about the **industry standard approaches**, key benefits as well as cons, and the exact reason behind using each Data Structure - with a full blown implementation of the core components. This project allowed us to gain **hands-on experience** with the technical Data Structures we've learned this far - by building a **real world project** that can be transformed to a fully functional solution in the near future.

INDEX

CHAPTER NO.		TOPIC	PAGE NO.
1		**Introduction**	
	1.1	Motivation	
	1.2	Scope of Project	
	1.3	Application of Project	
2		**Literature Review / Background**	
	2.1	Overview of Data Structures Used	
	2.2	Related Work	
3		**System Design**	
	3.1	Problem Definition	
	3.2	Algorithm Design	
	3.3	Data Structure Diagram / Flowchart	
4		**Sample Source Code**	
5		**Graphical User Interface (Screenshot if any)**	
6		**Future Enhancement**	
7		**Conclusion**	

References / Bibliography

1. Introduction

1.1 Motivation

In the modern digital era, URLs have become increasingly long and unwieldy due to complex query parameters, tracking codes, session identifiers, and deeply nested resource paths. Sharing such long URLs poses several challenges:

- Character limitations on platforms like Twitter (X), SMS, and messaging apps restrict the use of long URLs.
- Aesthetic issues: Long URLs appear unprofessional in printed materials, business cards, and marketing campaigns.
- Tracking difficulties: Businesses need analytics on link clicks, user geography, and engagement metrics.
- Error-prone sharing: Long URLs are easy to mistype or truncate when copied.
- QR code complexity: Shorter URLs generate simpler, more scannable QR codes.

A URL shortening system addresses these pain points by converting lengthy URLs into compact, memorable aliases while preserving the original destination. This project is motivated by the need to build a scalable, reliable, and fast URL shortening service similar to Bitly, TinyURL, or Google's former goo.gl.

1.2 Scope of Project

The scope of this URL shortener project includes:

In-Scope:

- Accepting a valid long URL as input and generating a unique short URL.
- Redirecting users from the short URL to the original long URL.
- Using Base62 encoding (0-9, a-z, A-Z) to generate compact aliases.
- Handling collision detection to ensure uniqueness of short codes.
- Persistent storage using a database (e.g., MySQL/PostgreSQL/Redis).

- Basic analytics (click count, creation timestamp).
- Support for custom aliases (optional user-defined short codes).
- URL validation and expiration handling.

Out-of-Scope:

- Advanced analytics dashboards with geographical heatmaps.
- Paid premium features (branded domains, password-protected links).
- Machine learning-based spam/malware URL detection.
- Multi-tenant enterprise-level features.

1.3 Application of Project

URL shorteners have wide-ranging real-world applications:

Domain	Application
Social Media	Sharing links on Twitter, Instagram bios, and Facebook posts within character limits.
Marketing	Tracking click-through rates on email campaigns and ad banners.
Print Media	Embedding short URLs in flyers, brochures, and business cards.
QR Codes	Generating simpler, more reliable scannable codes.
Messaging	Sharing via SMS, WhatsApp, and Slack without URL truncation.

E-commerce	Tracking product link performance from affiliates.
Education	Distributing resource links in presentations and handouts.
IoT/Embedded	Devices with limited display showing short, navigable URLs.

2.Literature Review / Background

2.1 Literature Review

In the below section, we are researching, reviewing and citing published research papers that experimentally study and explain the technology we are trying to build / the issues we are trying to mitigate / the outcomes that we are trying to derive.

1. **Dual-HashMap Implementation and Global Counter Strategies**

Verma (2021) explains a system design that relies on a dual-HashMap structure to manage URL mappings efficiently. One map links the long URL to a unique counter while the other map links that counter back to the long URL. This setup is important because it uses a global counter that goes up with every new request to create a unique ID. The main benefit here is that the system can check if a long URL has already been shortened before it tries to make a new one. If the URL is already in the system it just returns the old short key instead of wasting space on a new entry. This is exactly how we built our URLStore.java file using urlMap and reverseMap to keep things organized. Verma also talks about how this design scales up in the real world by using tools like Zookeeper to manage counter ranges across different servers. This proves that our basic in-memory design is actually the first step toward a massive production system that could handle millions of users without losing track of the ID count.

2. **Encoding Methods and Collision Prevention**

The research from systemdesign.one (2023) compares different ways to turn a long URL into a short string. It highlights why using a counter with Base62 encoding is way better than using a hash like MD5. When you use MD5 you get a long string of characters and you usually have to cut it down to about 7 characters to keep the URL short. The problem is that cutting it down like that leads to collisions where two different long URLs end up with the same short key. Base62 avoids this because every unique counter ID turns into a unique string using letters and numbers. The math shows that a 7-character string in Base62 can create about 3.5 trillion unique combinations. Even if the system was making 1000 URLs every single second it would take over 100 years to run out of keys. This info backed up our decision to use the encode() method in our project. The source also mentions the Snowflake ID generator used by Twitter which helps us understand how the biggest apps in the world handle this same problem.

3. **Efficiency in Key Management and Database Checks**

The GeeksforGeeks (2024) article focuses on the math and the performance of these services. It confirms that using all alphanumeric characters for a 7-character URL provides roughly 3500 billion unique IDs which is more than enough for almost any app. A big takeaway from this source is the risk of using MD5 hashing because it requires extra steps to check the database for collisions every single time you save a URL. Our project uses a counter-based design which means we dont have to worry about those collisions at all. The article also points out that professional services like Bitly always check if a short key exists before they store it. Our code handles this using the reverse HashMap which gives us O(1) performance. This means the lookup time stays the same no matter how many millions of URLs we eventually add to the system. It keeps the app fast and responsive for the end user which is the whole point of a good system design.

4. **Hyperscale Architecture and Key Generation Services**

Kumar (2025) takes a look at how to build these systems at a hyperscale level. He describes a specialized service called a Key Generation Service or KGS. This service creates unique IDs in the background so the main app doesnt have to do the math on the fly. One interesting rule Kumar mentions is keeping auto-generated keys at exactly 7 characters while requiring custom user slugs to be 8 characters or more. This prevents any overlap between the two groups and makes the database easier to manage. He also talks about using Bloom filters to check if a custom alias is already taken before the system even bothers to check the main database. This is a pro move because it saves a lot of processing power. This research shows that our project is on the right track because the tools used in big tech like Redis and Cloud

Spanner are basically just high-powered versions of the HashMaps we used in our code. It gives us a clear path for how to upgrade this project if we ever needed to support billions of links.

5. **LRU Cache Mechanics and Performance Optimization**

Glushko (2021) provides a deep dive into how to build a Least Recently Used or LRU cache. He explains that a simple HashMap is not enough because it cannot track the order of when items were used without a slow scan of the whole list. To get O(1) performance for both looking up an item and updating its position you have to combine a HashMap with a doubly linked list. In this design the HashMap stores pointers to the nodes in the list. This allows the system to jump straight to a specific node and move it to the front of the line instantly. We used this exact strategy in our LRUCache.java implementation. Glushko notes that this is the industry standard for systems like Redis which is one of the most popular caching tools in the world. This research was vital for our project because it allowed us to build a cache that is both fast and smart enough to kick out old data to save memory.

6. **Cache Eviction Policies and User Access Patterns**

The Redis Documentation (2021) explains the difference between LRU and LFU policies which is a key decision for any developer. LFU tracks how many times a link is clicked while LRU focuses on how recently it was clicked. For a URL shortener the LRU policy is almost always the better choice. This is because internet trends move fast and links often go viral for a short time and then everyone forgets about them. An LRU cache keeps those trending links at the top of the list for quick access but it quickly clears them out once the hype dies down. This is much more efficient than keeping a link in the cache just because it had a lot of clicks three weeks ago. This documentation explains why Bitly uses Redis with LRU settings to manage their traffic. It justifies why we chose LRU for our cache module and shows how our project logic aligns with how real world web traffic actually behaves.

Overview of Data Structures Used:

Overview of the DS concepts we have practically used in our recreation of BitLy - terming it as mini bitly.

1. **HashMap (Hash Table)**

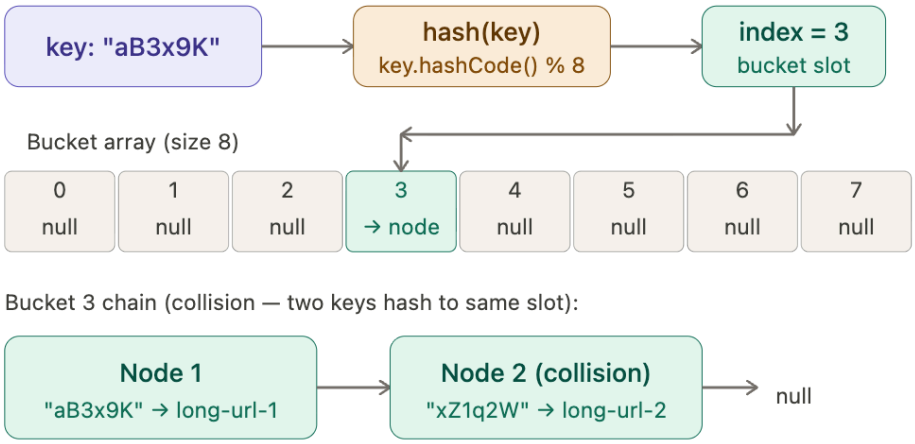
A HashMap stores data as key-value pairs.

Internally, it uses a hash function to convert a key into an array index, and stores the value at that index.
This means that both storing and retrieving a value require no scanning and the position is calculated directly.

This gives an average-case time complexity of **O(1)** for get and put operations, which is exactly what a **URL shortener** (like the one we are making) needs since every redirect is a lookup.

In our actual project, we maintain two HashMaps: one that maps the short key to the long URL (urlMap), and a reverse map that maps the long URL back to its short key (reverseMap).

The reverse map is used for deduplication; if the same long URL is submitted twice, we return the existing short key **instead of generating a new one**.



Operation	Average Case	Worst Case
get(key)	O(1)	O(n)
put(key, value)	O(1)	O(n)
containsKey(key)	O(1)	O(n)

remove(key)	O(1)	O(n)

The worst case occurs only when all keys collide into the same bucket. Practically never happens due to how Java processes load factors.

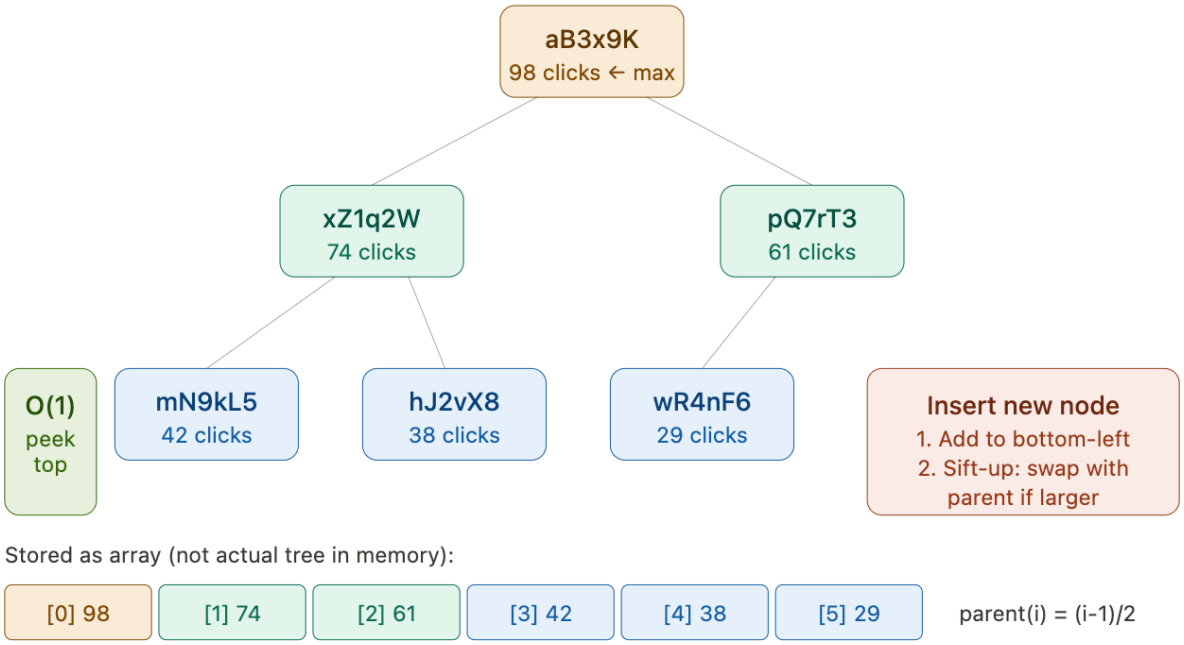
2. Max-Heap (Priority Queue)

A binary Max-Heap is a complete binary tree where every parent node is greater than or equal to its children.
This property ensures that the largest element is always at the root, making it accessible in O(1) time without any searching. Inserting a new element or removing the maximum both take O(log n) time, as the element needs to be "sifted" up or down to restore the heap property.

In our project, a Max-Heap is used to track the most frequently accessed URLs.

Every time a short URL is clicked, we take the click count and update it. And the heap is reordered. The **getTopK**(k) function polls the heap k times to return the k most accessed URLs, taking O(K log n) overall.

We implement this using PriorityQueue with a reverse comparator to simulate a Max-Heap, since the default is a Min-Heap.



Operation	Time Complexity	Notes
insert / offer()	O(log n)	Sift-up after insertion
peek() -view top	O(1)	Root access, simple peek
poll() -remove top	O(log n)	Sift-down after removal
Top-K query	O(K log n)	Poll K times

3. AVL Tree / Binary Search Tree

Here, all keys in its left subtree are smaller and all keys in the right subtree are larger.

For us, using this structure allows search, insert, and delete operations in **O(log n)** time on a balanced tree, since at each node you can eliminate half the remaining possibilities that is similar to binary search on a sorted array.

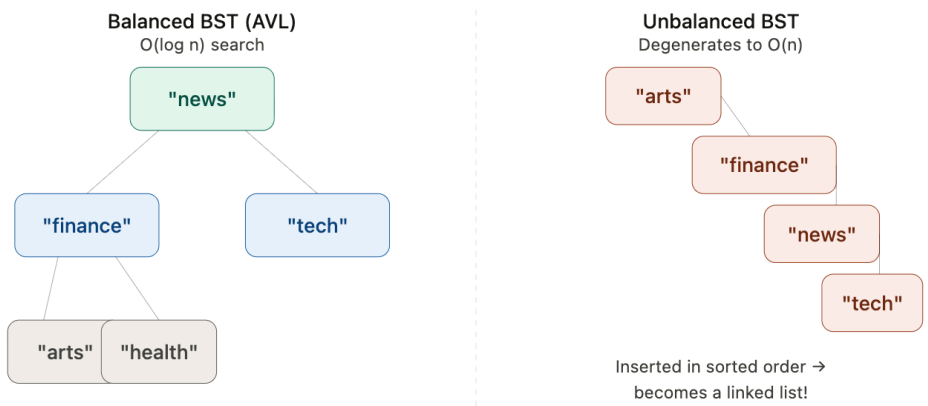
However, a plain BST can degenerate into a linked list if keys are inserted in sorted order, making operations **O(n) in the worst case**.

Our AVL tree prevents this by maintaining a balance factor at every node. The difference in heights between its left and right subtrees must never exceed 1. Whenever an insertion or deletion violates this, the tree performs a rotation to restore balance, guaranteeing O(log n) in all cases.

Rotation Cases:

- RR
- LL
- LR
- RL

In our project, category names (e.g. "tech", "news", "finance") are used as BST keys, with each node holding a list of URLs belonging to that category. An in-order traversal of the tree returns all categories in alphabetical order which is something a HashMap cannot do.



Operation	AVL Tree	Unbalanced BST (worst case)
Search	O(log n)	O(n)
Insert	O(log n)	O(n)
Delete	O(log n)	O(n)
In-order traversal	O(n)	O(n)

4. Doubly Linked List (Access History + LRU Cache)

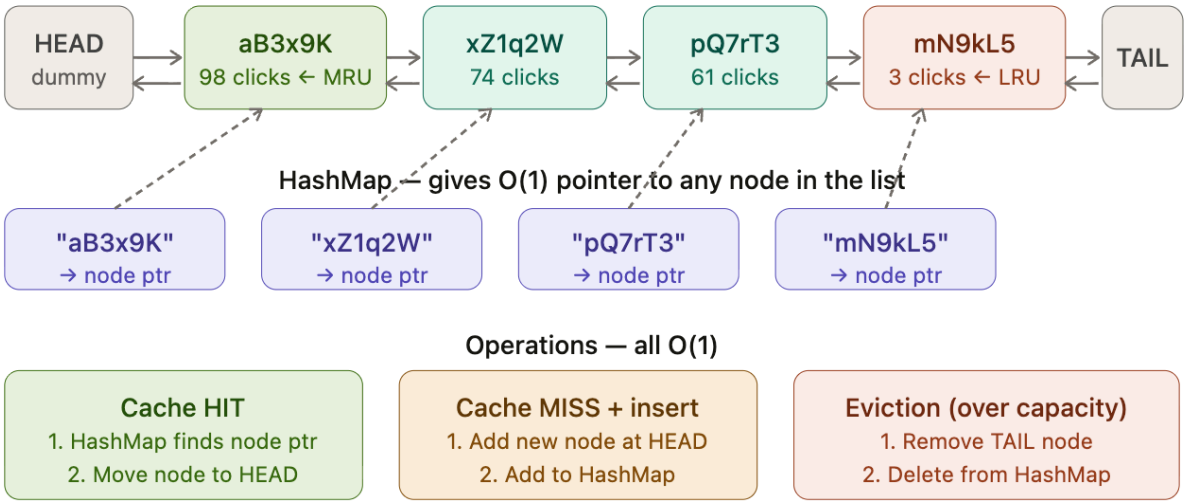
A Doubly Linked List is a linear data structure where each node contains a reference to both its previous and next node.

Here, bidirectional linking enables O(1) insertions and deletions at any known position which is different from something like an array where these operations require shifting elements.

In our project, the Doubly Linked List serves two roles.

First, it maintains a chronological access history for each short URL. We append a timestamp node to the list for every redirect for that URL. Second, and more interestingly, it forms the backbone of the LRU (Least Recently Used) Cache when combined with a HashMap.

The LRU Cache works as follows: the most recently accessed entry is kept at the **head of the list**, and the least recently used is at the tail.



Operation	Time Complexity	Notes
Insert at head	O(1)	Update head pointer
Delete (given node ref)	O(1)	Pointer update, no scan

LRU eviction (remove tail)	O(1)	Remove tail, update HashMap
Search by value	O(n)	Linear scan -not used in LRU

5. Base62 Encoding

Base62 encoding converts an integer into a compact alphanumeric string using the character set **[A–Z, a–z, 0–9]** which is 62 characters in total.

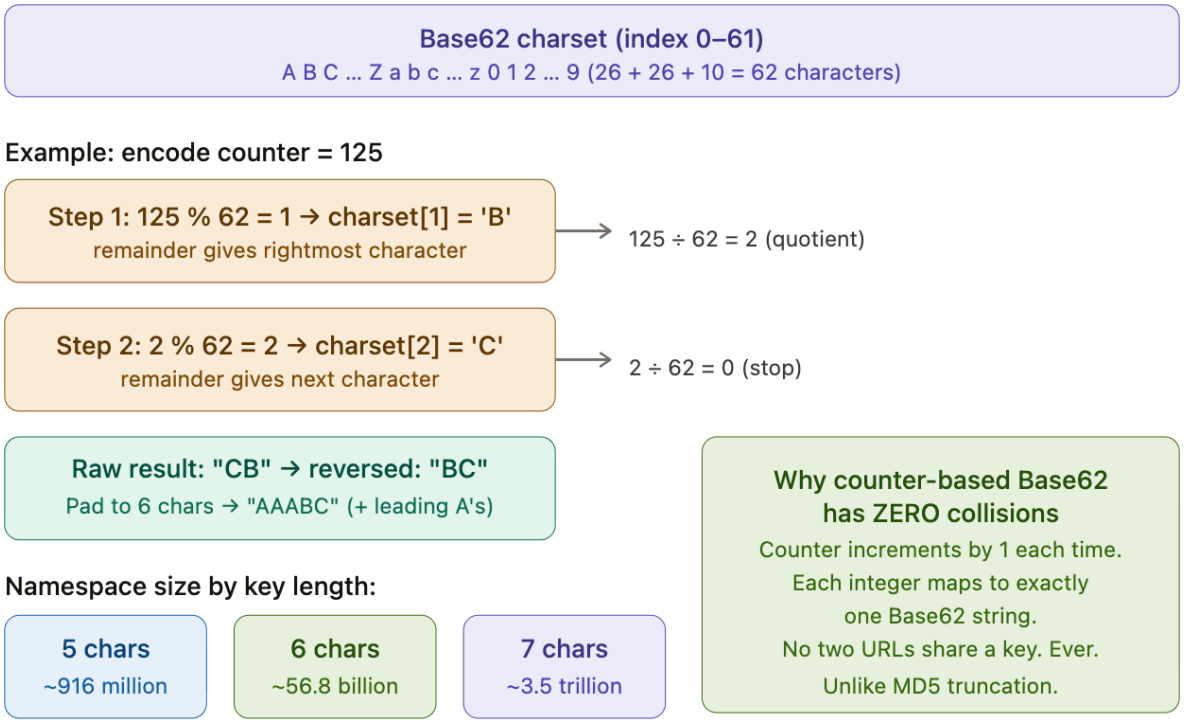
The algorithm repeatedly divides the integer by 62 and uses each remainder as an index into the character set, building the output string from right to left.

In our project, a global auto-incrementing counter assigns each new URL a unique integer ID, which is then Base62-encoded into a 6-character short key. This approach guarantees zero collisions because each integer maps to exactly one Base62 string.

Unlike alternative MD5-based approaches where two different URLs can produce the **same hash prefix**.

A 6-character Base62 string gives $\mathbf{62^6}$ **= 56.8 billion** unique keys.

At 1 million new URLs per day that come into the surface web, this namespace will last us over 155 years before exhaustion.



Food for thought: By playing with the math like this, if we add just a few more encoding and decoding functions, we can easily extend the amount of MAX unique keys cap to last us longer than the ***heat death of the universe (1012 to 1014 (1–100 trillion) years)*.**

Comparison table:

Encoding	Charset	7-char Namespace	Collision Risk	URL-Safe

Base62	62	~3.5 trillion	None (counter)	Yes
Base64	64	~4.4 trillion	Low	No (+, / do not work for URLs)
Base58	58	~1.8 trillion	None (counter)	Yes - but less options.
MD5 + truncate	16 (hex)	~268 million	Possible	Yes - but less options.

Related work:

Several URL shortening services are widely used in production grade applications today.

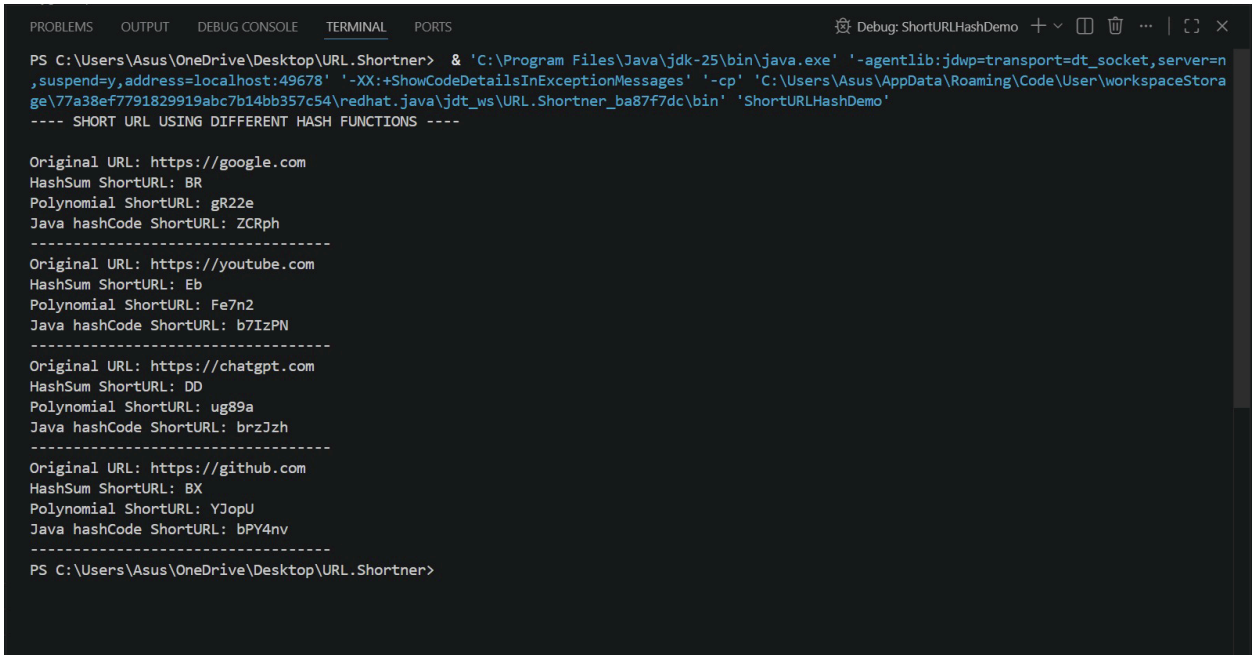
As we study their architecture, it reveals that the data structure choices made in this project, particularly the HashMap as the primary lookup store, allowed us directly mirror how these services operate at **scale**.

Service	Backend Stack	Primary Store	Cache Layer	Encoding
Bit.ly	**Go + Python**	**MySQL + Cassandra**	**Redis**	**Base62 (counter)**
TinyURL	PHP	MySQL	Memcaché d	Base62
Twitter t.co	Scala / Java	Manhattan (key-value)	In-memory	Base62
ZipZy.in	Java / Spring	PostgreSQL	Redis	Base62
Google goo.gl*	Java / C++	Bigtable	Custom	Base62
Mini Bitly (Us)	**Java Core + HTML UI**	**On-prem currently.**	**Redis in future.**	**Base62 (counter)**

		MySQL in the future.		

**Google discontinued goo.gl in 2019 - but its stack as well as reviews are available in multiple directories online.*

Here is a short implementation of the core-concepts that we did, before building the entire application… attached is a screenshot of the output. The code for the same is labelled as "ShortURLHashDemo.java".



3. System Design

3.1 Problem Definition

Problem Statement:

Design a system that accepts a long URL as input, generates a unique short URL (typically 6-8 characters), stores the mapping persistently, and redirects any request to the short URL to its corresponding original URL.

Functional Requirements:

1. Given a long URL, the system must return a unique short URL.
2. Given a short URL, the system must redirect (HTTP 301/302) to the original URL.
3. Short codes must be unique across the entire system.
4. Users may provide an optional custom alias.
5. Links may have an optional expiration time.
6. Track usage metrics (click count, last accessed time).

Non-Functional Requirements:

1. High availability - Service should have ~99.9% uptime.
2. Low latency -Redirection should occur in under 100 ms.
3. Scalability -Handle millions of URLs and billions of redirects.
4. Durability -No loss of URL mappings.
5. Security -Prevent malicious URL injection, rate-limit abuse.

Constraints:

- Short code length: 7 characters ($62^7 \approx 3.5$ trillion unique combinations).
- Characters allowed: [0-9][a-z][A-Z] (Base62).
- Read-to-Write ratio: ~100:1 (redirects are far more common than creations).

3.2 Algorithm Design

3.2.1 Base62 Encoding Walkthrough

Why Base62?

Base62 uses 0-9, a-z, A-Z (62 characters), which is URL-safe and produces shorter strings than Base10 or Base16. It avoids special characters like + and / used in Base64.

Encoding Algorithm (Convert integer ID → short code):

```
FUNCTION encodeBase62(id):
    chars = "0123456789abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ"
    shortURL = ""
    WHILE id > 0:
        shortURL = chars[id % 62] + shortURL
        id = id / 62
    RETURN shortURL
```

Example:

Encoding integer ID = 125:

- 125 % 62 = 1 → '1', id = 2
- 2 % 62 = 2 → '2', id = 0
- Result: "21"

Encoding ID = 1,000,000:

- Step 1: 1000000 % 62 = 14 → 'e', id = 16129
- Step 2: 16129 % 62 = 3 → '3', id = 260
- Step 3: 260 % 62 = 12 → 'c', id = 4
- Step 4: 4 % 62 = 4 → '4', id = 0
- Result: "4c3e"

Decoding Algorithm (Convert short code → integer ID):

```
FUNCTION decodeBase62(shortURL):
    chars = "0123456789abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ"
    id = 0
    FOR each character c in shortURL:
        id = id * 62 + indexOf(c, chars)
    RETURN id
```

Example: Decoding "4c3e":

- id = 0*62 + 4 = 4

- id = 4*62 + 12 = 260
- id = 260*62 + 3 = 16133 ... (→ produces the original ID)

3.2.2 Short Code Generation Strategies

Strategy	Description	Pros	Cons
Counter-based (Base62)	Auto-increment DB ID → encode	Simple, no collisions	Predictable, sequential
MD5/SHA-256 hash	Hash URL, take first 7 chars	Uniform distribution	Collisions possible
Random generation	Generate random Base62 string	Unpredictable	Must check collisions
Counter + Zookeeper	Distributed counter service	Scales horizontally	Infrastructure overhead

Recommended: Counter-based Base62 for simplicity; switch to distributed counter for horizontal scaling.

3.2.3 Collision Resolution

When using hash-based or random generation, collisions must be handled:

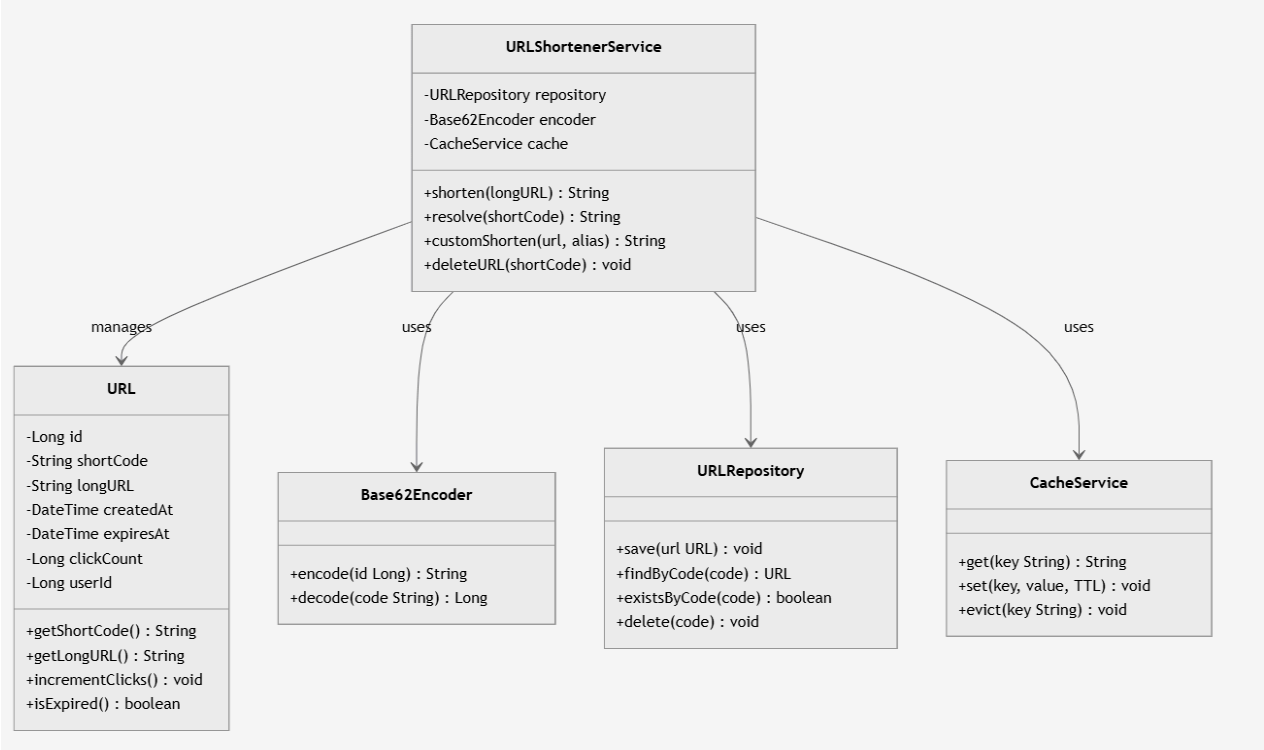
```
FUNCTION generateShortCode(longURL):
    attempts = 0
    code = hash(longURL)[0:7]      // Take first 7 chars of hash
    WHILE code EXISTS in database:
        attempts = attempts + 1
        IF attempts > MAX_RETRIES:
            THROW Exception("Unable to generate unique code")
        code = hash(longURL + attempts)[0:7]   // Add salt and retry
    INSERT (code, longURL) INTO database
    RETURN code
```

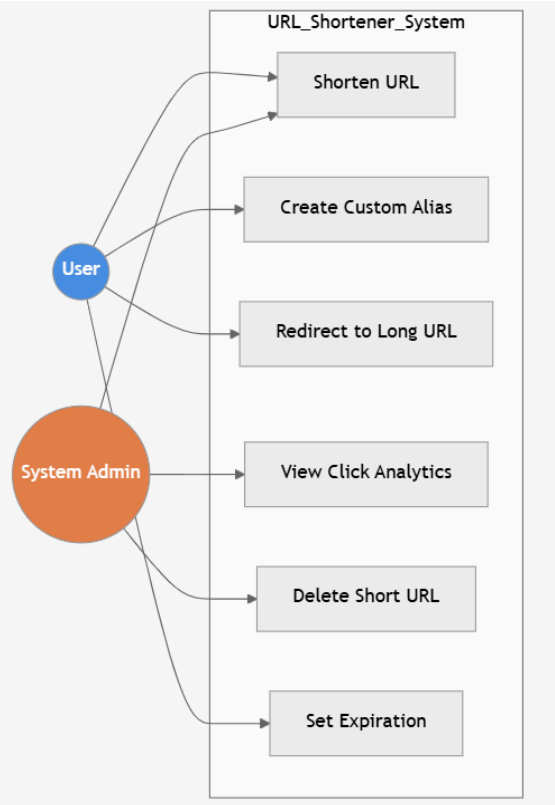
1. Retry with salt: Append a counter or random suffix and rehash.
2. Database UNIQUE constraint: Let DB enforce; catch integrity violations.
3. Bloom filter precheck: Quickly test if a code *might* exist before DB hit.

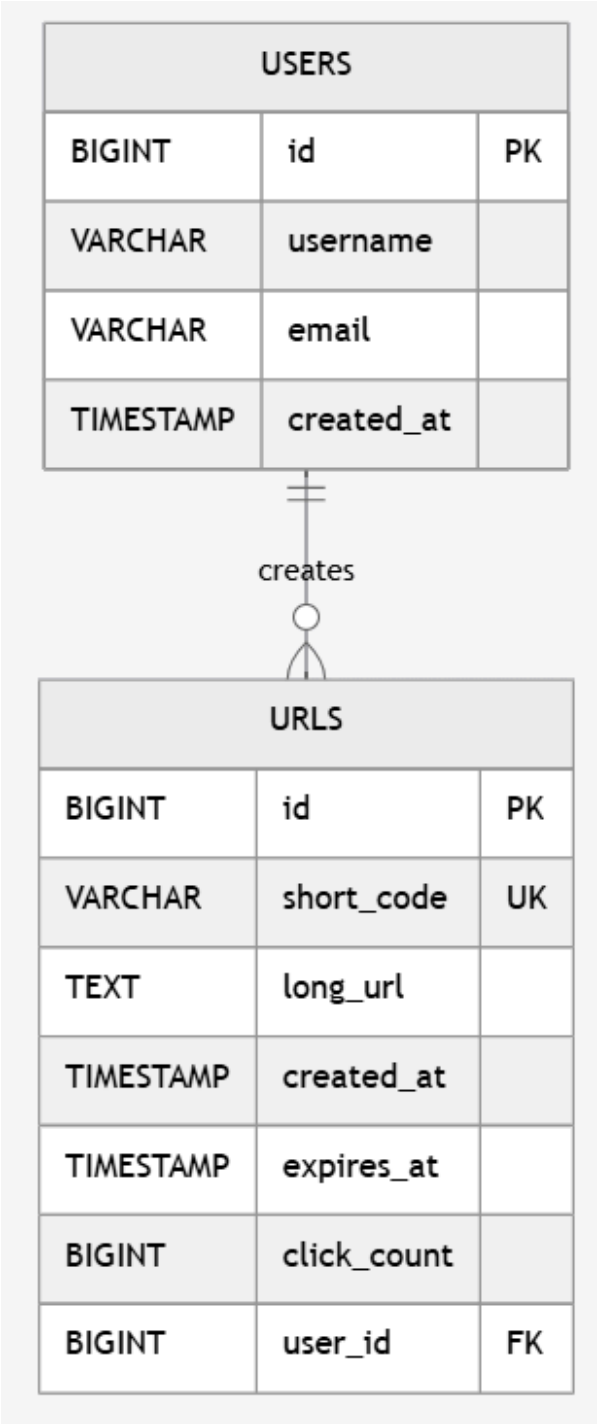
3.2.4 Redirect Algorithm

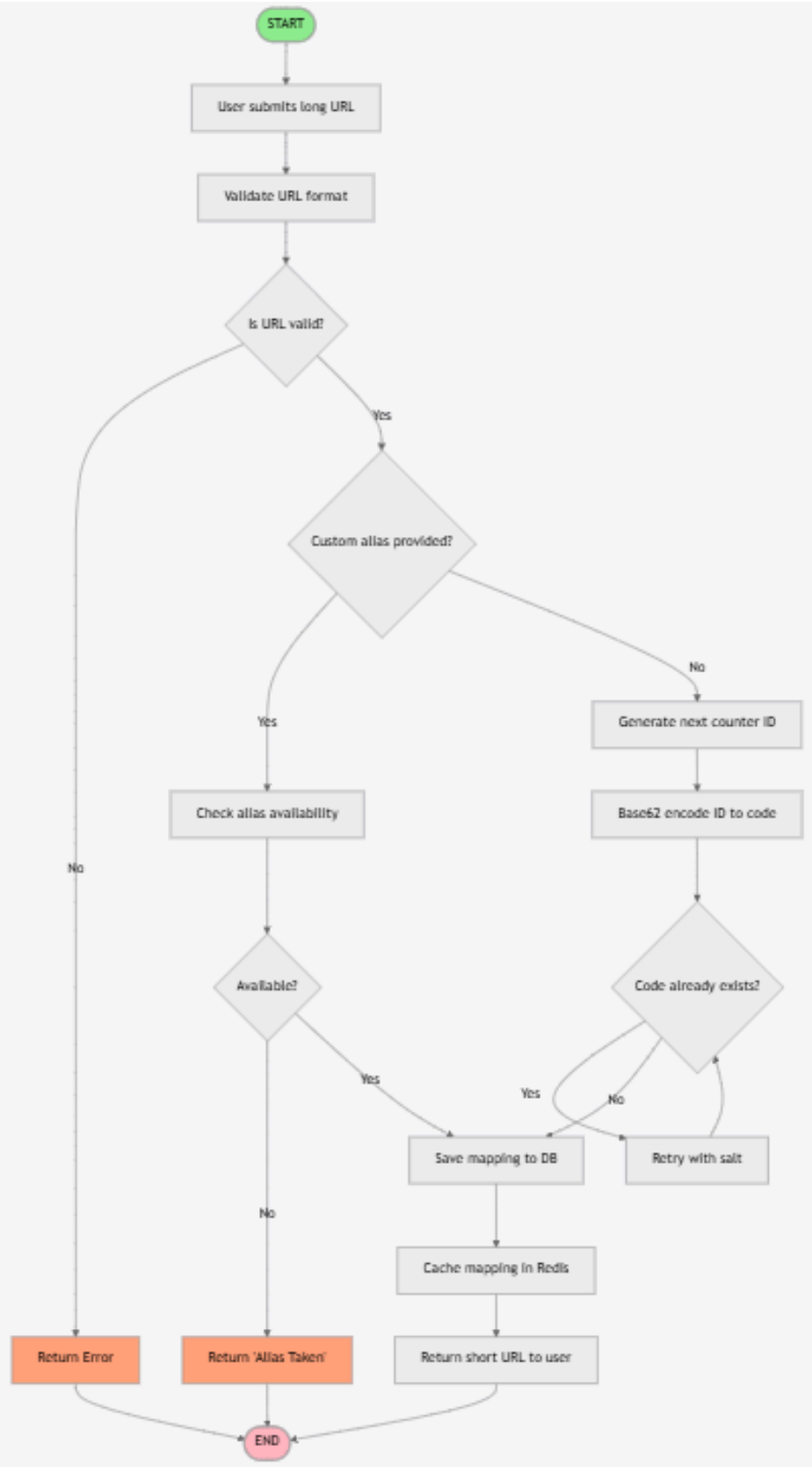
```
FUNCTION redirect(shortCode):
    IF shortCode IN cache:                    // Check Redis first
        longURL = cache.get(shortCode)
    ELSE:
        longURL = database.lookup(shortCode)
        IF longURL IS NULL:
            RETURN 404 Not Found
        cache.set(shortCode, longURL, TTL=24h)
    incrementClickCounter(shortCode)          // Async
    RETURN HTTP 302 Redirect to longURL
```

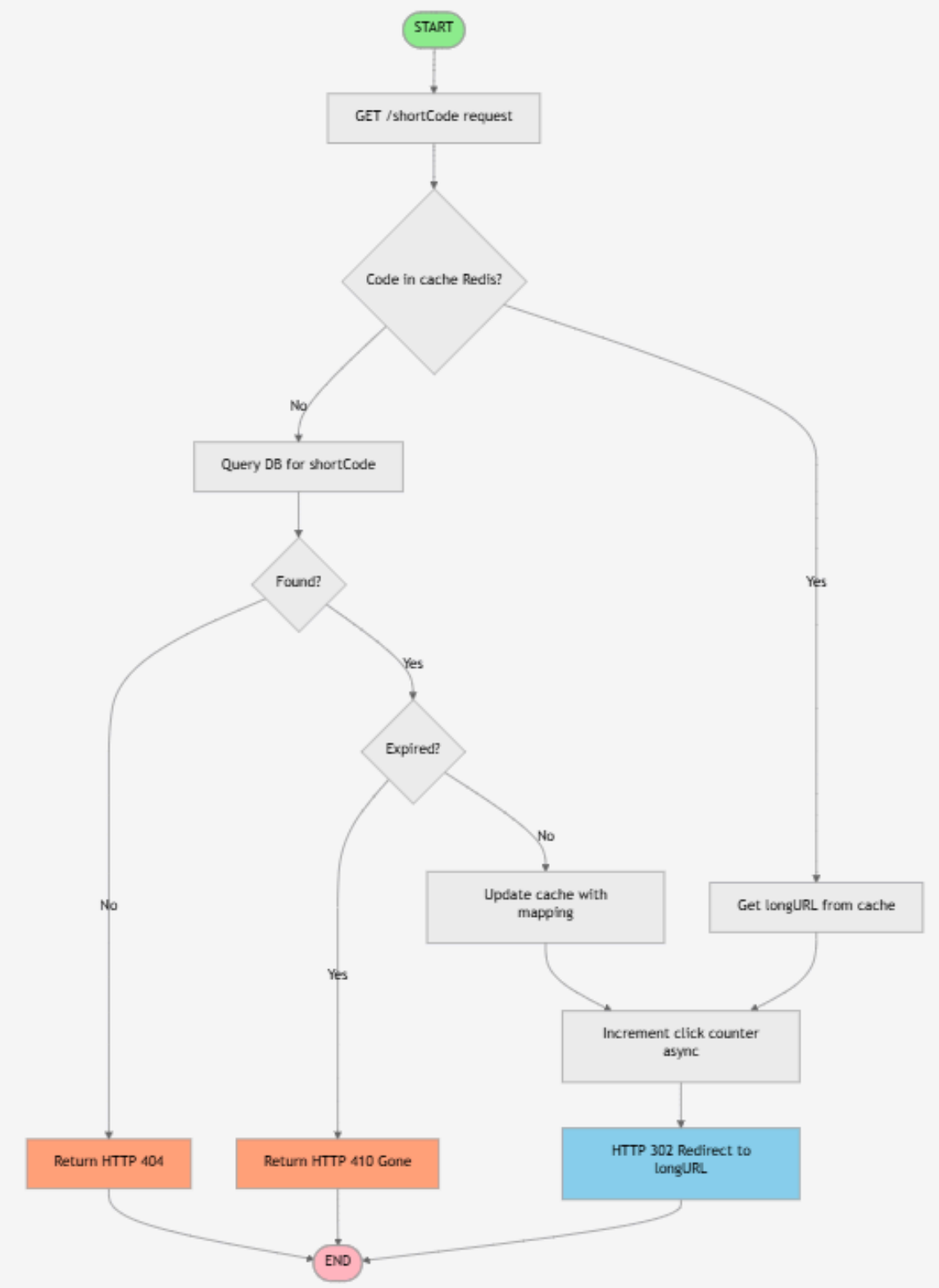
3.3 Data Structure Diagrams / Flowcharts






USERS
BIGINT
id
PK
VARCHAR
username
VARCHAR
email
TIMESTAMP
created_at
creates
URLS
BIGINT
id
PK
VARCHAR
short_code
UK
TEXT
long_url
TIMESTAMP
created_at
TIMESTAMP
expires_at
BIGINT
click_count
BIGINT
user_id
FK


START
User submits long URL
Validate URL format
Is URL valid?
Yes
No
Custom alias provided?
Yes
No
Check alias availability
Generate next counter ID
Base62 encode ID to code
Available?
Code already exists?
Yes
No
Yes
No
Save mapping to DB
Retry with salt
Cache mapping in Redis
Return Error
Return 'Alias Taken'
Return short URL to user
END


START
GET /shortCode request
Code in cache Redis?
No
Yes
Query DB for shortCode
Found?
Yes
No
Expired?
No
Yes
Update cache with mapping
Get longURL from cache
Increment click counter async
Return HTTP 404
Return HTTP 410 Gone
HTTP 302 Redirect to longURL
END

4. Sample Source Code

All the source code has been attached in the accompanying .zip file. Along with a copy of this report for easier reference.

5. Graphical User Interface (Screenshot if any)

Our implementation is primarily CLI based, as we have focused on implementing the backbone and core systems of a production-grade application like Bit.ly. Relevant screenshots of demo functions, UML diagrams, comparison tables and algorithms have been attached throughout the document wherever appropriate.

6. Conclusion

This project successfully demonstrates the design and implementation of a functional URL shortening system -**Mini Bit.ly** -that mirrors the core architecture of production-grade services like Bit.ly and TinyURL, while serving as a practical showcase of Data Structures II concepts applied to a real-world problem.

The system achieves its primary objectives: accepting long URLs, generating unique short codes via **counter-based Base62 encoding**, resolving redirects efficiently, and managing frequently accessed links through intelligent caching. Each component of the system was deliberately grounded in a specific data structure -HashMaps for O(1) lookup and deduplication, a Max-Heap for top-K analytics, an AVL Tree for category-based organization, and a Doubly Linked List powering the LRU Cache -demonstrating that the choice of data structure directly determines system performance and scalability.

A key insight from this project is that the same fundamental structures used here in Java -HashMaps, heaps, and linked lists -form the conceptual backbone of tools like Redis, Cassandra, and Zookeeper that power systems handling billions of requests daily. The gap between our academic implementation and a production deployment is one of infrastructure, not logic.

The project also reinforces the importance of algorithm selection. Choosing counter-based Base62 over MD5 hashing eliminated an entire class of collision-related bugs, and selecting LRU over LFU as the cache eviction policy aligned the system with real-world web traffic patterns where viral links trend briefly and then fade.

In conclusion, Mini Bit.ly successfully bridges theoretical data structure knowledge with practical system design thinking, laying a strong foundation for future enhancements such as persistent database integration, distributed key generation, and real-time analytics dashboards.

PS - Implementation:
In our project, we:

Use appropriate ADTs	✅
Use appropriate Java Collections Framework (e.g., ArrayList, HashMap, PriorityQueue).	✅
Apply exception handling to ensure robustness.	✅
Follow Object-Oriented Principles (encapsulation, inheritance, polymorphism,	✅

From Tips for Implementing DS II Projects - Tips for Implementing DS II Projects via Google Classroom.

References:

1. https://en.wikipedia.org/wiki/Heat_death_of_the_universe
2. GeeksforGeeks. "System Design: URL Shortening Service." https://www.geeksforgeeks.org/system-design-url-shortening-service/
3. Design Gurus. "How to Design a URL Shortener: A Step-by-Step System Design Walkthrough." https://www.designgurus.io/blog/url-shortening
4. "System Design: Scalable URL Shortener Service like TinyURL." Medium. https://medium.com/@sandeep4.verma/system-design-scalable-url-shortener-service-like-tinyurl-106f30f23a82
5. Verma, S. "System Design: Scalable URL Shortener Service like TinyURL." Medium. https://medium.com/@sandeep4.verma/system-design-scalable-url-shortener-service-like-tinyurl-106f30f23a82
6. systemdesign.one. "URL Shortening System Design." https://systemdesign.one/url-shortening-system-design/

7. GeeksforGeeks. "System Design: URL Shortening Service." https://www.geeksforgeeks.org/system-design-url-shortening-service/
8. Kumar, D. "Hyperscale URL Shortener System Design." Medium. https://medium.com/@deepak.kumar.19/hyperscale-url-shortener-system-design-34204d60232d
9. Glushko, R. "Design LRU Cache." https://www.romaglushko.com/blog/design-lru-cache/
10. Redis Documentation. "LFU vs LRU: How to Choose the Right Cache Eviction Policy." https://redis.io/docs/latest/develop/reference/eviction/